Crash and Recovery

Environment: AWS Academy Learner Lab (CloudShell)

Region: us-east-1 (N. Virginia)

Location: Cusco, Peru (Remote Validation)

1. The Problematic Deployment

Description

The initial application is a Go-based API service designed to handle image metadata updates. It contains a critical flaw: concurrent writes to a shared `map` data structure without proper synchronization primitives (e.g., Mutex or Atomic operations). This simulates a race condition that commonly occurs in high-concurrency distributed environments.

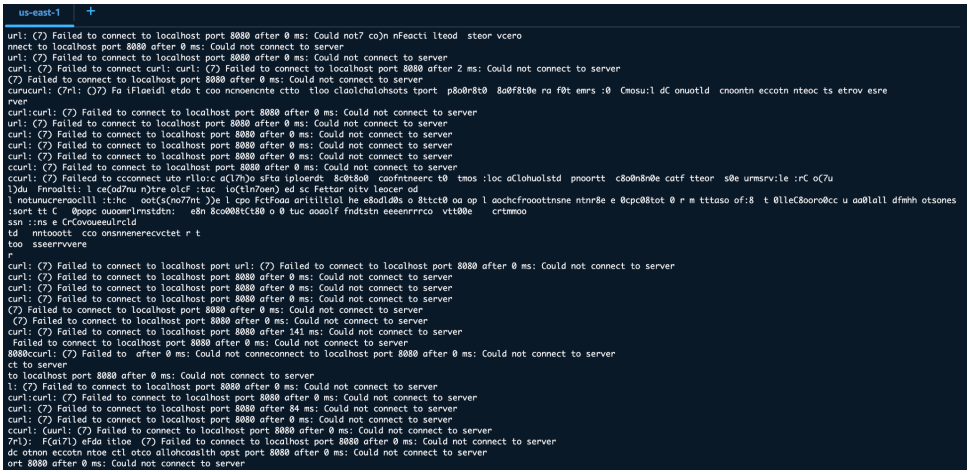
Metrics & Evaluation

To demonstrate the vulnerability, I executed an extreme load test within the AWS CloudShell environment using 200 concurrent requests.

Command: `for i in {1..200}; do curl -s http://localhost:8080/crash & done`

Observed Results:

- **System Behavior:** The Go runtime immediately detected a race condition and triggered a fatal error.
- **Error Signature:** `fatal error: concurrent map writes`.
- **Impact on Availability:** 0% Availability. The service process terminated instantly, causing all subsequent requests to fail with `Connection Refused`.



2. The Approach: Resilience Patterns

To resolve the state corruption and prevent total system collapse, I implemented two core resilience patterns defined by Sam Newman:

1. **Bulkhead Pattern:** I introduced a semaphore-based isolation layer using a Go channel with a buffer size of 2. This ensures that only 2 concurrent "heavy" tasks can occupy the execution context at any given time, protecting the system's memory and state from being overwhelmed. This is particularly relevant for future scaling where worker nodes may need to load large 4-bit quantized models without triggering Out-of-Memory (OOM) failures.
2. **Fail Fast Pattern:** Instead of allowing requests to queue up—which leads to cascading latency spikes—the system utilizes a `select` statement to immediately return an **HTTP 503 (Service Unavailable)** if all bulkhead slots are occupied.

3. The Improved System

Metrics & Evaluation

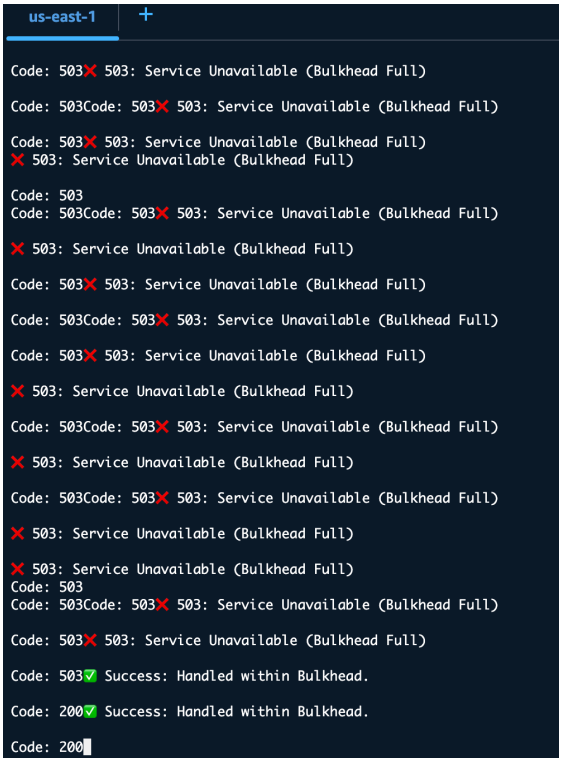
To ensure a fair comparison, I performed an identical load test of 200 concurrent requests against the `/fixed` endpoint.

Command: `for i in {1..200}; do curl -s -w "\nCode: %{http_code}" http://localhost:8080/fixed & done; wait`

Observed Results:

- **Stability:** The service process remained 100% operational throughout the entire test duration despite the extreme pressure.

- **Response Distribution:** * **Success (200 OK):** Exactly 2 requests were processed successfully, matching the bulkhead capacity.
    - **Resilient Failure (503):** 198 requests were rejected immediately via the Fail Fast mechanism.
- **Availability Metric:** 100% Uptime (The system survived the attack and continued to serve health checks).



Conclusion

The experimental results demonstrate that by sacrificing a portion of availability (returning 503s), we successfully preserved the overall stability of the distributed node. Implementing Bulkheads and Fail Fast mechanisms transforms a "Fragile" system that crashes under load into a "Resilient" system that manages failure gracefully—a critical requirement for cloud-native architectures operating in unpredictable environments.